In Real-time Scene-aware Collaborative 3D Editor

Salaets Florian

Software Systems Engineering Technology

1 Introduction

Game engines such as **Unity** and **Unreal** serialise scenes into large text or binary files. Version control surfaces these as **unresolvable merge conflicts**. Text and 2D editors solved this with real-time sync built on **Operational Transformation** (Ellis & Gibbs, 1989) or **Conflict-free Replicated Data Types** (Shapiro et al., 2011). Both guarantee strong eventual consistency, but they differ in how much of each user's **intent** survives (Sun & Ellis, 1998).

A 3D scene is a hierarchical **scenegraph**: setting an absolute position is not commutative, but a relative movement delta is (Preguiça et al., 2018; Zhou et al., 2023). The **granularity** at which conflicts are detected determines how much intent survives. No prior work compares strategies across the full range of scenegraph conflict types in a single, controlled environment — this research closes that gap.

Research questions

- Q1 Do all four strategies guarantee strong eventual consistency?
- Q2 For which conflict types does each strategy preserve both collaborators' intended changes, and where does **intent loss** occur?
- Q3 Does **property-level** conflict detection (Variant B) eliminate the loss seen under object-level LWW (Variant A)?
- Q4 Does representing movement as **commutative deltas** (Variant C) reliably preserve the accumulated effect of concurrent movement operations?
- Q5 What are the practical differences between the **OT** approach (Variant D) and the **CRDT**-based approaches (A–C) in a 3D scenegraph context?
- Q6 Are these approaches a realistic way of working in the intended context, the **game industry**? Can they serve as a viable or better alternative to existing file-based workflows?

2 Materials & Methods

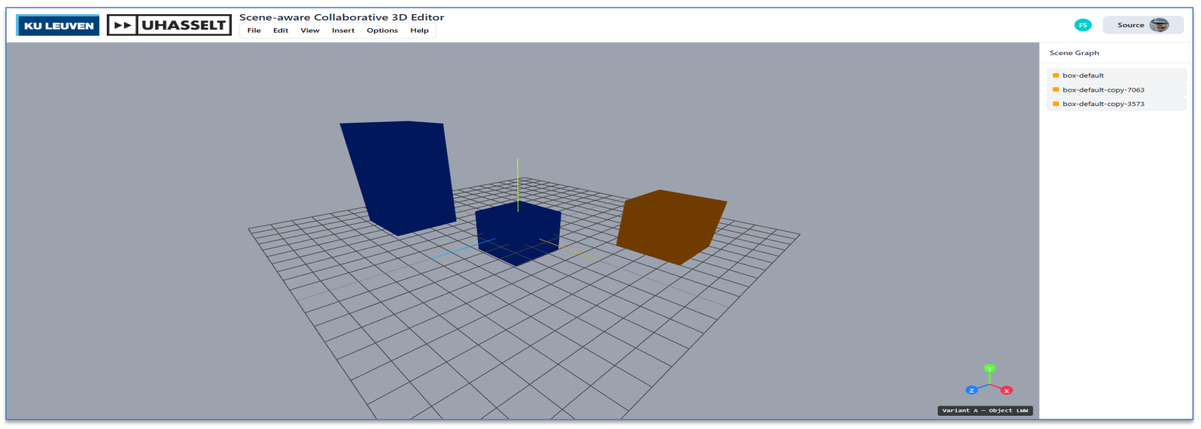

Figure 1: The collaborative 3D editor interface

All four variants implement the same Collaboration-Strategy **interface** in TypeScript 5.9 (addObject, updateObject, moveObject, removeObject, linkObject, unlinkObject, subscribe). The application layer is variant-agnostic, meanin that the same test suite runs against every implementation.

A — Object-level LWW

Single Y.Map<SceneObject> Each write replaces the whole object, this means that concurrent edits on different properties still collide.

B — Property-level LWW

Nested Y.Map<Map> Each scalar (position_x, color…) has its own key, meaning that only same-attribute edits collide.

C — Operation log + deltas

Append-only Y.Array<Op> Movement stored as commutative deltas rather than absolute values.

D — Operational Transformation

Authoritative in-memory server. Each op carries a clientRevision; server transforms ops against unseen log entries.

Fifteen scenarios were implemented in Vitest 4.0. Two simulated clients (Alice, Bob) share a scene; **disableSync()** pauses replication while both act independently, then **enableSync()** reconnects them. Two metrics are recorded: **(M1) Convergence** — all clients reach an identical state — and **(M2) Intent preservation** — how many intended property changes survive.

3 Results & Discussion

All four variants pass **convergence** on every scenario, satisfying strong eventual consistency (Shapiro et al., 2011). Convergence is necessary but not sufficient — two systems can converge to the same wrong outcome. **Table 1** shows how **intent preservation** differs.

Conflict scenario	A	B	C	D
Same-property edit S1, S12	LWW	LWW	LWW	1st wins
Different property, same object S2, S8	1 lost	both	both	both
Four disjoint properties S14	2 lost	all 4	all 4	all 4
Repeated movement deltas S4, S15	1 lost	1 lost	accumulate	1 lost
Edit on deleted object S3, S9	no-op	no-op	no-op	no-op
Concurrent reparent S5, S10	LWW	LWW	LWW	LWW

Table 1: Outcome of each strategy per conflict family across 15 scenarios

The largest difference appears in **different-property conflicts** (S2, S8, S14). Variant A treats the whole object as one merge unit, so an Alice-moves-while-Bob-recolors edit collapses to one full-object write — half the intent is gone. Variants B, C and D resolve at property granularity and preserve both edits.

Delta commutativity (S15). Alice moves an object +1·X three times while offline, Bob moves it +1·X twice. Ideal: **+5** Variants A, B and D submit absolute positions — one final write overwrites the other, yielding either +3 or +2. Variant C appends each move as a delta to the shared log. Addition is commutative, so order does not matter and all five moves always accumulate to +5.

Granularity ladder. Bigger merge units cause more collateral loss. The contention zone shrinks from whole file → whole object (A) → single property (B, D) → nothing for commutative ops (C).

4 Conclusion

Q1 - Convergence

All four converges

Every variant satisfies SEC across 15 scenarios. A–C inherit it from Y.js; D from server broadcast.

Q2 / Q3 - Granularity

Property & object

B, C and D preserve all changes when properties don't overlap (S2, S8, S14). A loses one client's edit on every collision.

Q4 - Deltas

Only C accumulates moves

Commutative delta operations carry zero risk of loss for repeated movements. A, B and D all degrade to LWW.

Q5 – OT vs CRDT

D = inspectable rules

D matches B for most scenarios, with the policy explicit in transform() rather than implicit in the data structure.

Q6 No variant blocks concurrent editing, no file or object is ever locked. The contention zone shrinks from a whole file (traditional) to whole object (A), single property (B, D) or nothing for delta moves (C). All approaches are a viable real-time alternative to Preforce-style file locking.

Supervisors / Co-supervisors / Advisors: Put Jeroen

Ellis, C. A., & Gibbs, S. J. (1989). Concurrency Control in Groupware Systems. *ACM SIGMOD Record*, 18(2), 399–407.
Sun, C., & Ellis, C. A. (1998). Operational transformation in real-time group editors. *Proc. ACM CSCW 1998*.
Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). Conflict-free Replicated Data Types. *INRIA Research Report 7686*.
Preguiça, N., Baquero, C., & Almeida, P. S. (2018). Conflict-free Replicated Data Types: An Overview. *arXiv:1806.10254*.
Zhou, S., Li, Y., Shang, T., & Kong, W. (2023). A paradigm for collaborative 3D editing via list CRDTs. *Proc. CSAE 2023*, ACM.
Sun, C., Xia, S., Sun, D., Chen, D., Shen, H., & Cai, W. (2012). Transparent adaptation of single-user applications for multi-user real-time collaboration. *ACM TOCHI*